Service Tiers Toolkit

Three parts: (1) intake form that sorts people into tiers, (2) full setup guide for Type 3 (no access for you), (3) DIY starter kit for Type 4. Commands and prices change, so check the current docs for each tool before you deploy.

Part 1: Intake Form

How to use it

Sit with the person for five minutes, or send the form ahead. Ask the questions as written. Don't explain the tiers until they have answered, so you get honest preferences rather than the "right" answer.

The form

Name / household: _____ Date: _____

A. What are we protecting?

1. Which phones and computers do you use? (make, model, Android or iPhone)
   _____
2. Roughly how many photos and videos do you have? Under 50 GB / 50 to 200 GB / 200 GB to 1 TB / more than 1 TB / no idea
3. Where do your photos live today? Google Photos / iCloud / Mega / only on the phone / other: _____
4. What documents would hurt most to lose? (taxes, ID, medical, school, legal, other)
   _____
5. Has anyone in your household lost a phone, been locked out, or lost data before?
   _____

B. How much control do you want?

6. Would you be comfortable letting me handle your phone or backups directly? Yes / Somewhat / No
7. Are you okay with me technically being able to see your files, if I promise not to look? Yes / Only if I can see how it's protected / No
8. Do you want to run this yourself, with my help only if you get stuck? Yes / No / Not sure

9. If you forgot your backup password, would you accept that the data could be lost for good, in exchange for me having no way to open it? Yes / No

C. Practicalities

10. Is there a spot at home for a small always-on device and a drive? Yes / No / Not sure
11. Do you have a password manager? Yes (which) / No
12. Who would you trust to help your family if you were unable to? _____
13. Anything you want kept private even from me? _____

Scoring

If they say…	Tier
Q6 Yes, Q7 Yes	**Type 1:** Trusting
Q7 "Only if I can see how it's protected"	**Type 2:** Secure, access OK
Q7 No, Q9 Yes	**Type 3:** No access for you
Q7 No, Q9 No	**Type 2** with an encrypted vault for sensitive files, or explain the Type 3 trade-off again
Q8 Yes	**Type 4:** DIY
Q8 Yes plus unusual requirements (multiple sites, hardware keys, estate planning)	**Type 5:** bespoke project

Q13 matters for anyone in Type 1 or 2: offer an encrypted vault (Cryptomator) for those files.

After the form

1. Explain the recommended tier in two sentences and confirm they agree.
2. Fill in the matching agreement (Type 3 needs the modified clauses below).
3. Add them to your tracking sheet: tier, devices, estimated size, replica drive yes/no, date seeded, last restore drill.
4. Schedule the setup session.

Part 2: Type 3 Setup Guide (Zero Access for You)

The principle

Data is encrypted on the person's device **before** it reaches you, with a key only they hold. You provide space, uptime and offsite replication of ciphertext. If you are breached, hacked, subpoenaed or simply curious, there is nothing readable to find.

What you give up: you can't restore for them, can't confirm that files open, and can't recover a lost key. Say this clearly at intake and again in writing.

Architecture

```
Phone  -> Ente app (E2EE) ----------------> Your Ente server (ciphertext only)
Phone  -> Syncthing-Fork --> Their home device (holds the key)
Home device -> restic (their key) -------> Your append-only repo (ciphertext)
                                            \-> Your offsite copy (still
ciphertext)
Home device -> local drive (their own copy, encrypted with LUKS)
```

Three layers: Ente for photos, Syncthing and restic for documents, and a home device that owns the keys. Pick the pieces that fit the household.

Part A: Photos with Ente

Option 1: Ente's hosted service. Easiest, with no server for you. Their data sits with Ente, encrypted. You help with setup but host nothing.

Option 2: Self-hosted Ente (you host the ciphertext).

1. Deploy Ente's server on a small VM or LXC using the current quickstart in Ente's self-hosting documentation. It runs the API server, a Postgres database and an S3-compatible object store (MinIO or similar).
2. Put it behind Tailscale, or behind Caddy with CrowdSec if you expose it publicly.
3. On each phone, install the Ente app. Before signing up, open the app's developer settings (tap the logo on the welcome screen repeatedly) and set the server endpoint to your URL. Then create the account.
4. Ente shows a **recovery key** (a list of words). The person writes it down. It goes on their break glass sheet. You never see it.
5. Turn on backup for the camera folders and any albums they care about.

Important: Ente is sync, not backup. A deletion in Ente propagates (it has a trash with a retention period). So add:

- **Server-side snapshots of the ciphertext.** Back up the object store and database with restic to B2 or Hetzner. You can't read it, but it protects against deletion, corruption and your own mistakes. To restore, you rebuild the server and they sign in with their key.
- **A plain-file export they control.** The Ente command-line tool can export a library to ordinary files. Run it on their home device on a schedule to an encrypted local drive. This is their escape hatch if Ente or your server disappears.

Limits to tell them: no server-side machine-learning search the way Immich has, and import from other services is limited.

Part B: Documents

Option 1: Their computer or home device runs restic to your repo (recommended).

On your server, run restic's REST server in append-only mode so a compromised client can't delete history:

```
sudo apt install restic
rest-server --path /srv/restic --append-only --private-repos --htpasswd-file
/etc/rest-server/.htpasswd
```

Create a user per household:

```
htpasswd -B /etc/rest-server/.htpasswd anna
```

Expose it over Tailscale only. On the client:

```
export RESTIC_REPOSITORY=rest:http://anna:PASSWORD@your-server:8000/anna/
export RESTIC_PASSWORD_FILE=/home/anna/.restic-pass
restic init
restic backup /home/anna/Documents
```

The restic password is **theirs**. It is encrypted on their device before upload, and you only ever see opaque blobs.

- **Schedule it** with a systemd timer or cron on the home device, so it runs nightly with no action from them.

- **Pruning:** append-only mode blocks `forget --prune`, so the repo only grows. Every few months, take the repo out of append-only mode for a short window, have them run `restic forget --keep-daily 7 --keep-weekly 4 --keep-monthly 12 --prune`, then turn append-only back on. Put it in your calendar.

Option 2: Cryptomator vault synced to storage you provide. Simpler for non-technical people. Files are encrypted inside a vault folder on their computer, and that folder syncs to your server or any cloud. Weaker versioning, but easy to understand.

Option 3: Proton Drive. An end-to-end encrypted service you don't host. Easy on both Android and iPhone. Add it as a second copy rather than your only one.

Documents from phones:

- **Android:** Syncthing-Fork syncing the Documents folder to the person's home device (a Pi or mini PC), which then runs restic. You are not in the sync path at all.
- **iPhone:** iOS makes folder sync difficult. Use a scan workflow into a Cryptomator vault or Proton Drive, and tell them honestly that this part needs a little habit.

Part C: The home device

This is the piece that owns the keys.

- **Hardware:** a Raspberry Pi 5 or N100 mini PC, plus a USB or SATA drive larger than their documents and exports.
- **Disk encryption:** LUKS on the external drive, so a burglar gets nothing.
- **Jobs it runs:** Syncthing (receiving from the Android phone), restic to your repo, and the Ente CLI export.
- **Remote access for you:** install Tailscale and give yourself SSH access **only if the person agrees**, because this is a path to their keys. For a strict Type 3, don't take that access; use a written runbook and a video call.
- **Updates:** unattended-upgrades on, so it stays patched without help.

Part D: Keys and recovery

1. **Recovery keys on paper**, one per tool: the Ente recovery key and the restic password. Both go on the break glass sheet in the safe.
2. **A second copy** in a sealed envelope with a trusted person, or split with Shamir's Secret Sharing for the careful.
3. **Never store the keys with you** if the point of the tier is that you can't read the data.

4. **Test the keys:** at setup, have them sign in to Ente on a spare phone using only the recovery key, and restore one file from restic on another computer using only the written password.

Part E: What you can monitor without reading anything

You can see timestamps and sizes without seeing contents. This script runs on your server and alerts if a household's repo hasn't changed in a week:

```
#!/bin/bash
for d in /srv/restic/*/; do
  newest=$(find "$d" -type f -printf '%T@\n' | sort -n | tail -1)
  age=$(( ($(date +%s) - ${newest%.*}) / 86400 ))
  if [ "$age" -gt 7 ]; then
    curl -d "No backup from $(basename $d) in $age days"
https://ntfy.sh/YOUR-PRIVATE-TOPIC
  fi
done
```

Run it daily from cron. Then message the person: "Your backup hasn't run in a week. Is the home device on?" You can check freshness without being able to verify content.

Part F: Restore drills (the person does them, with you on call)

Twice a year:

1. They pick five random files and restore them from restic to a temporary folder.
2. They open each file.
3. They sign in to Ente on a different device and confirm recent photos appear.
4. They record the date and result on the break glass sheet.

If they can't, you help them troubleshoot **without taking their key**.

Part G: Changes to the agreement

Use the standard agreement with these edits:

- **Section 4 (Privacy):** replace with: "Data is encrypted on the Participant's devices before it reaches the Host. The Host does not hold the encryption keys and is technically unable to read the Data."

- **Section 5 (Restoring):** replace with: "The Host cannot restore the Data. The Host will provide storage, retrieve encrypted data on request, and assist the Participant in restoring using their own keys."
- **Add a Key Loss clause:** "If the Participant loses all copies of their encryption keys and recovery keys, the Data cannot be recovered by anyone, including the Host."
- **Exit Procedure:** delivery is a copy of the encrypted repository plus instructions. The Participant already holds the keys, so the Host doesn't need to produce plain files.

Onboarding checklist for Type 3

- Intake form complete; trade-offs (no restore by you, key loss is permanent) explained and acknowledged
- Home device installed, drive encrypted, Tailscale and updates configured
- Ente account created against your server (or hosted), recovery key written down
- restic repo initialised, password written down, first backup complete
- Syncthing running (Android) or scan workflow explained (iPhone)
- Server-side snapshots of ciphertext verified
- Password manager and break glass sheet completed
- Test restore done together, by them
- Agreement signed with the Type 3 edits
- Calendar entries: monthly stale check, twice-yearly restore drill, repo prune window

Part 3: DIY Starter Kit (Type 4)

Hand this section to someone who wants to run everything themselves. Replace the bracketed items. It assumes a single household, basic Linux comfort and about a weekend of setup.

What you will build

A small always-on home server that receives your photos and documents automatically, keeps them safe from disk failure, and copies them (encrypted) to two offsite locations. It tells you when something breaks.

```
Phones -> Immich (photos) + Syncthing (documents)
            |
       Home server (mirrored drives, UPS)
            |-- restic nightly --> Backblaze B2   (offsite 1)
            |-- restic nightly --> second provider (offsite 2)
            |-- monthly rotation --> offline drive (kept elsewhere)
            |
       Healthchecks.io pings tell you if any job stops
```

Shopping list (verify prices)

- **Server:** an N100 mini PC with 16 GB RAM, or a 2-bay NAS (Synology or similar). Avoid single-board computers for the main library if you can.
- **Storage:** two matching drives (CMR, NAS-rated) in a mirror, sized at least 2x your current library. Add one extra external drive for the offline copy.
- **UPS:** a small one with USB so the server shuts down cleanly in a power cut.
- **Offsite:** a Backblaze B2 account, plus a second provider (Hetzner Storage Box, or a relative's machine).
- **Domain:** optional, only if you plan to expose services publicly. Tailscale avoids this.

Step 1: Base system

1. Install Debian or Ubuntu Server on the boot drive.
2. Create the mirror (ZFS mirror or Linux RAID 1) and mount it at `/srv/data`.
3. Enable disk encryption on the data drives if the server could be stolen.
4. Turn on automatic security updates: `sudo apt install unattended-upgrades`.
5. SSH keys only: set `PasswordAuthentication no` in `/etc/ssh/sshd_config`.
6. Install Docker and Docker Compose from Docker's official repository.
7. Install Tailscale and sign in, so you can reach everything remotely without opening ports.
8. Install NUT or your UPS vendor's tool, and test that it triggers a clean shutdown.

Step 2: Photo ingest with Immich

Use Immich's official Docker Compose file and `.env` from the current Immich installation guide. Don't copy old compose files from forums, because Immich changes quickly.

- Set the upload location to `/srv/data/immich`.

- Create the admin account, then add a user for each family member.
- On each phone, install the Immich app and set the server URL (your Tailscale address).
- Android: turn off battery optimisation for the app. iPhone: open the app regularly, because iOS limits background uploads.
- **Database dumps:** keep Immich's automatic database backup enabled, and confirm the dump files land inside the folder you back up in Step 5.

Step 3: Documents with Syncthing

Create `/srv/data/syncthing/docker-compose.yml`:

```
services:
  syncthing:
    image: syncthing/syncthing:latest
    container_name: syncthing
    hostname: home-server
    environment:
      - PUID=1000
      - PGID=1000
    volumes:
      - /srv/data/syncthing:/var/syncthing
    ports:
      - 8384:8384
      - 22000:22000/tcp
      - 22000:22000/udp
    restart: unless-stopped
```

Then:

1. Open the web interface at port 8384 (over Tailscale), and set a username and password straight away.
2. Install **Syncthing-Fork** on Android. Add the server as a device and share the Documents folder.
3. Set the phone's folder type to **Send Only**.
4. On the server, enable **Staggered File Versioning** (or Trash Can) for the folder, so a deletion on the phone doesn't erase the server's copy.
5. iPhone has no official Syncthing app. Use a scanner app that saves into a folder you can sync, or use Nextcloud for documents.

Step 4: Why snapshots matter

If you use ZFS, schedule snapshots (for example with `sanoid`): daily for 7 days, weekly for 4, monthly for 12. They protect against accidental deletion and ransomware on the server itself, and they cost very little space. On other filesystems, restic (below) fills this role.

Step 5: Offsite backups with restic

Install restic (`sudo apt install restic`). Create a password file that only root can read:

```
sudo sh -c 'openssl rand -base64 32 > /root/.restic-pass'
sudo chmod 600 /root/.restic-pass
```

Write that password down on paper. Without it the backup cannot be opened.

Initialise the Backblaze B2 repository (create a bucket and an application key limited to that bucket first):

```
export B2_ACCOUNT_ID=your-key-id
export B2_ACCOUNT_KEY=your-application-key
export RESTIC_PASSWORD_FILE=/root/.restic-pass
restic -r b2:your-bucket-name:home init
```

Create `/usr/local/bin/backup.sh`:

```
#!/bin/bash
set -euo pipefail

export RESTIC_PASSWORD_FILE=/root/.restic-pass
export B2_ACCOUNT_ID=your-key-id
export B2_ACCOUNT_KEY=your-application-key
HC_URL=https://hc-ping.com/YOUR-UUID

curl -fsS -m 10 --retry 3 "$HC_URL/start" > /dev/null

restic -r b2:your-bucket-name:home backup /srv/data/immich
/srv/data/syncthing --exclude-caches

restic -r b2:your-bucket-name:home forget --keep-daily 7 --keep-weekly 4 --
keep-monthly 12 --keep-yearly 5 --prune
restic -r b2:your-bucket-name:home check --read-data-subset=2%

curl -fsS -m 10 --retry 3 "$HC_URL" > /dev/null
```

```
sudo chmod 700 /usr/local/bin/backup.sh
```

Schedule it with a systemd timer. Create `/etc/systemd/system/backup.service`:

```
[Unit]
Description=Offsite backup

[Service]
Type=oneshot
ExecStart=/usr/local/bin/backup.sh
```

And `/etc/systemd/system/backup.timer`:

```
[Unit]
Description=Nightly backup

[Timer]
OnCalendar=*-*-* 02:30:00
Persistent=true

[Install]
WantedBy=timers.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now backup.timer
```

Second offsite copy: repeat with a different provider and its own repository (for example a Hetzner Storage Box over SFTP: `restic -r sftp:user@host:home init`). Two independent providers means one account problem can't take out both.

Make one copy immutable: enable Object Lock on the B2 bucket, or give the backup job a key that can add but not delete. A compromised server then can't wipe the history.

Step 6: The offline copy

Once a month, plug in the spare drive, copy the library to it (`rsync -a /srv/data/ /mnt/offline/`), unplug it and store it away from home, at work or with a relative. Put a reminder in your calendar. Encrypt the drive with LUKS so a lost drive is harmless.

Step 7: Monitoring

1. Create a free account at Healthchecks.io and a check with a schedule of daily and a grace period of a few hours. Put its ping URL in `backup.sh`.
2. Send alerts to your phone through ntfy, email or Telegram.
3. Add a SMART check: install `smartmontools` and enable the daemon so failing drives email you.
4. Add a free-space alert: a cron job that warns above 85 percent usage.
5. The rule: **silence must never mean fine.** Every job should either ping or alert.

Step 8: Restore drill (every quarter)

```
export RESTIC_PASSWORD_FILE=/root/.restic-pass
export B2_ACCOUNT_ID=your-key-id
export B2_ACCOUNT_KEY=your-application-key

restic -r b2:your-bucket-name:home snapshots
restic -r b2:your-bucket-name:home restore latest --target /tmp/restore-test
--include /srv/data/syncthing
```

Then open ten random files, including a photo, a PDF and a video. Once a year, rebuild the whole setup on a spare machine from nothing but the paper password and the offsite repo. If you can't, fix the process now rather than during a real failure.

Step 9: Security basics

- No ports open to the internet. Use Tailscale. If you must expose something, use the Caddy and CrowdSec guide.
- Two-factor authentication on the cloud accounts, the domain registrar and your email.
- Use a password manager, and keep the restic password on paper in a safe place.
- Update the server and the containers monthly. Read Immich release notes before upgrading.

One-page runbook (print this)

Where things are: server at [location]; data in `/srv/data`; compose files in `[path]`.

Credentials: all in [password manager]; restic password on paper at [location].

Routine: monthly update and offline-drive rotation; quarterly restore drill; yearly full-rebuild test.

If the server dies: buy a replacement, install Debian, install restic, restore from B2 using the paper password, bring the containers back up from the compose files.

If a drive fails: replace it, resilver the mirror, check SMART on the other drive.

If I am unavailable: [trusted person] has the sealed envelope with the passwords and this runbook.

DIY checklist

- Hardware bought and assembled; UPS tested
- Mirror created; data encrypted if needed; updates automatic; SSH keys only
- Tailscale running
- Immich and Syncthing working from every device; versioning on
- restic jobs to two providers; one immutable copy
- Offline drive rotated and stored elsewhere
- Healthchecks, SMART and disk-space alerts working
- Restore drill passed; runbook printed
- Password manager, break glass sheet and recovery codes completed

Optional bridge back to you

If the DIY person wants an extra offsite copy without giving you access, they can point a second restic repository at storage you provide (an append-only REST server over Tailscale, as in Part 2). They keep the password; you hold only ciphertext. You can then offer a monthly stale-backup nudge without ever reading anything.

Part 4: App Data and Online Accounts

A condensed version of the full App Data Backup Menu (the separate doc, with the complete app-by-app reference, automation recipes and printable checklist). Use this section as the quick reference during onboarding.

Why this part exists

The camera-roll and folder backup in Parts 1 to 3 only captures what is saved to the phone as ordinary files. Data that lives inside an app, or in an online account the app merely displays, needs a different method.

Add this to the intake form

Question 14: Which apps or accounts hold information you couldn't replace? (Hand over the printable menu and let them tick.)

Decision guide

For each ticked item, ask in this order:

1. **Is it in an online account?** Pull it from the account on your server (rclone, IMAP, CalDAV, scheduled Takeout). Set up once.
2. **Is it only on the phone?** Use the app's own export, or an encrypted backup to a computer.
3. **Is it a one-off record** (statements, tax papers)? Make it an annual paperwork day.
4. **Is it unrecoverable?** (Signal on iPhone without a backup, some authenticators.) Say so plainly and agree what to do.

Codes: A = automatable after one login; S = semi-automatic (schedule expires or needs renewal); M = manual each time; X = no real export.

Cheat sheet: the most common requests

Item	Method	Code
Gmail	IMAP sync to your server, or Google Takeout	A or S
Google Drive, Dropbox, OneDrive	rclone with a deleted-file backup folder	A
Google Photos, YouTube, Keep	Google Takeout (scheduled every 2 months for a year, then renew)	S
Contacts and calendar	CardDAV and CalDAV sync, or vCard and ICS export	A or M
iCloud Photos	icloudpd on the server (no Advanced Data Protection), or the Apple data request	A with limits

Item	Method	Code
WhatsApp	Encrypted chat backup plus per-chat export to Files	M
iMessage	Encrypted backup to a computer	M
Signal	Android chat backup file; check current iPhone options	M or X
Facebook, Instagram, Messenger	Meta Accounts Center: Download your information	M
TikTok, X, LinkedIn, Snapchat	Each service's data download request	M
Apple Health	Health app: Export All Health Data	M
Notion, Evernote, Trello	Built-in workspace or board export	M
Password manager	Encrypted export, stored separately from its password	M or A
Authenticator codes	Printed recovery codes per account; move to a manager that stores codes	M or X
Bank and tax records	Annual download of PDFs and CSVs into the Documents folder	M

The whole-phone safety net (iPhone)

An **encrypted backup to a Mac or PC** catches much of the app data in one go, including Health data and saved passwords. Record its password on the break glass sheet. It is a snapshot, so schedule a monthly reminder.

Where exports go

```
/srv/data/households/<name>/exports/<app>/<date>/
```

Keep the previous export, record any encryption key, include the folder in the offsite jobs, and put sensitive exports (health, finance, private messages) in an encrypted vault, especially for Type 2 and 3 households.

Tracking table (one per household)

App	Where data lives	Method	Code	How often	Who does it	Lands in	Last done	Next due

Suggested cadences

- Automated pulls: nightly, with a monitoring ping.
- Chat and social exports: every 3 to 6 months, and before any phone change.
- Health and fitness: every 6 months.
- Statements and tax records: monthly or yearly.
- Google Takeout schedule: recreate yearly.

Tell them what can't be saved

Some apps have no export, exports are snapshots rather than live copies, formats may not restore back into the app, and services change their tools over time. Say this at the menu stage, not after a loss.

Type-specific notes

- **Type 1 and 2:** you run the pulls and keep the tracking table.
- **Type 3:** the person runs the exports and stores them in their own encrypted vault. You provide space and reminders only.
- **Type 4:** give them the menu and the automation recipes, and let them do the rest.